

BANCO OFICIAL DE 10 PROYECTOS INTEGRADORES

Ingeniería de requerimientos predefinida · Backend Java 21/Spring Boot · PostgreSQL · React + TypeScript · React Native + TypeScript · Docker · GitHub/CI · Spring AI

Documento	Aplicación
Tipo	Guía oficial de asignación y ejecución de proyectos por equipo
Equipos	3 a 5 estudiantes, conformados bajo conducción docente
Arquitectura	Monolito modular con arquitectura hexagonal simplificada
Evaluación	4 entregas progresivas, todas con documentación + avance MVP web/móvil
Regla central	La división interna de tareas no limita la responsabilidad individual sobre el proyecto completo

1. Propósito del banco de proyectos

Este documento define diez proyectos integradores de complejidad comparable para la asignatura Programación Aplicada. Cada proyecto se presenta como un encargo realista cuyo proceso de levantamiento de requerimientos ya fue realizado con el cliente. Por tanto, los equipos no deben “inventar” el problema ni reducir el alcance unilateralmente: deben aprender a interpretar, modelar, implementar, probar, documentar y defender una especificación previamente establecida.

El objetivo pedagógico es que todos los equipos recorran el mismo tipo de ingeniería: comprensión del dominio, modelado de datos, backend, API REST, arquitectura, frontend web, aplicación móvil, pruebas, Docker, integración continua y una función acotada de IA cuando corresponda. La diversidad está en el dominio; la exigencia técnica es equivalente.

2. Reglas generales de ejecución

- El docente asignará un proyecto a cada equipo. No se permite cambiar el problema central sin autorización.
- Cada proyecto será desarrollado durante toda la asignatura y se dividirá en cuatro partes evaluables.
- Todas las entregas incluyen documentación y software ejecutable; ninguna entrega puede consistir sólo en documentos o sólo en código.
- Desde la primera parte debe existir evidencia de web y móvil. El grado de completitud aumenta en cada entrega.
- El backend será un monolito modular con arquitectura hexagonal simplificada. No se requieren microservicios.
- PostgreSQL es la base de datos transaccional obligatoria. El esquema debe evolucionar mediante migraciones.
- Web: React + TypeScript. Móvil: React Native + TypeScript. Backend: Java 21 + Spring Boot.
- El equipo puede repartir tareas, pero cada estudiante debe comprender y poder defender el alcance completo exigible en el corte de evaluación.
- Las reglas críticas deben implementarse en el backend y respaldarse, cuando corresponda, con restricciones de base de datos.
- La función de Spring AI es complementaria y nunca puede sustituir validaciones deterministas o reglas de negocio.

3. Estructura uniforme de las cuatro entregas

Parte	Instancia	Foco	Regla de producto
1	PARTE I - Parcial 1	problema, requisitos, datos y backend fundacional	Documentación + incremento funcional web + incremento funcional móvil + backend + evidencia técnica
2	PARTE II - Trabajos Prácticos Integradores	flujo transaccional principal	Documentación + incremento funcional web + incremento funcional móvil + backend + evidencia técnica
3	PARTE III - Parcial 2	integración web/móvil, seguridad y calidad	Documentación + incremento funcional web + incremento funcional móvil + backend + evidencia técnica

4	PARTE IV - Examen Final	MVP integrado, IA responsable y defensa	Documentación + incremento funcional web + incremento funcional móvil + backend + evidencia técnica
---	-------------------------	---	---

La delimitación específica de cada proyecto aparece dentro de su ficha. El docente puede fijar fechas y ponderaciones administrativas, pero no debe eliminar la progresión técnica: cada corte debe demostrar una evolución verificable del mismo sistema.

4. Artefactos documentales obligatorios comunes

- Visión y alcance
- Stakeholders y actores
- Requisitos funcionales numerados
- Requisitos no funcionales numerados
- Reglas de negocio numeradas
- Historias de usuario/casos de uso y criterios de aceptación
- Modelo conceptual, DER lógico y diccionario de datos
- Arquitectura y estructura modular
- Contratos REST/DTOs/errores
- Matriz de trazabilidad
- Plan y evidencias de prueba
- Guía de instalación
- Manual de usuario
- Registro de decisiones técnicas
- Retrospectiva final y limitaciones

5. Requisitos no funcionales comunes para los diez proyectos

ID	Área	Requisito
RNF-01	Usabilidad	La interfaz debe mantener navegación consistente, mensajes claros de error, estados de carga y validaciones visibles.
RNF-02	Seguridad	Autenticación y autorización por roles; contraseñas nunca en texto plano; backend valida permisos independientemente del frontend.
RNF-03	Integridad	PostgreSQL debe aplicar PK, FK, UNIQUE, NOT NULL, CHECK e índices razonables; las reglas críticas no dependen sólo de la UI.
RNF-04	Mantenibilidad	Backend como monolito modular con arquitectura hexagonal simplificada; código organizado por módulos/casos de uso, no por carpetas genéricas masivas.
RNF-05	Calidad	Pruebas unitarias para reglas críticas y pruebas de integración para repositorios/endpoints esenciales.
RNF-06	Trazabilidad	Operaciones relevantes deben registrar usuario, fecha y cambio de estado cuando aplique.
RNF-07	API	API REST versionada o claramente prefijada; códigos HTTP correctos; DTOs de entrada/salida y errores consistentes.

RNF-08	Web	React + TypeScript; evitar any salvo justificación; componentes reutilizables; formularios tipados y consumo de API centralizado.
RNF-09	Móvil	React Native + TypeScript; navegación, autenticación, consumo de API y al menos un flujo transaccional completo.
RNF-10	DevOps	Docker Compose debe levantar al menos PostgreSQL y backend; CI debe compilar, probar y validar backend y frontend.
RNF-11	Configuración	Secretos y configuración sensible mediante variables de entorno; no deben versionarse credenciales reales.
RNF-12	Documentación	README ejecutable, decisiones arquitectónicas, modelo de datos, catálogo de API, pruebas y manual de despliegue.
RNF-13	Rendimiento	Listados con paginación cuando el volumen pueda crecer; consultas sin N+1 evidente en flujos críticos.
RNF-14	Observabilidad	Logging estructurado básico, manejo global de errores y correlación suficiente para diagnosticar una petición.
RNF-15	IA responsable	Si se implementa Spring AI, la función debe ser complementaria, auditable, con fallback y sin sustituir reglas deterministas.

01. ClinixFlow - Gestión Integral de Consultorio y Agenda Médica

Campo	Definición
Código	PA-01
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Una red pequeña de consultorios privados necesita ordenar la agenda, pacientes, profesionales, atenciones, antecedentes administrativos y seguimiento posterior. Hoy utiliza hojas de cálculo, mensajes por WhatsApp y registros dispersos. El cliente exige una solución web para recepción/administración y una aplicación móvil para pacientes y profesionales.

B. Objetivo contractual del producto

Construir un sistema transaccional que gestione pacientes, profesionales, especialidades, agendas, citas, estados de atención, notas de seguimiento no clínicas, recordatorios y reportes operativos, con trazabilidad completa y controles de acceso por rol.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Recepcionista	Opera los procesos diarios habilitados para su función y registra información transaccional.
Profesional de salud	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Paciente	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Identidad y perfiles
- Pacientes
- Profesionales y especialidades
- Disponibilidad y agenda
- Citas y estados
- Atención/seguimiento administrativo
- Notificaciones
- Reportes y panel
- Auditoría

E. Reglas de negocio obligatorias

RN-01. Una cita debe pertenecer a un paciente, un profesional y una franja horaria válida.

RN-02. No puede existir solapamiento de citas activas para el mismo profesional.

RN-03. Las citas siguen estados: SOLICITADA, CONFIRMADA, EN_ESPERA, ATENDIDA, CANCELADA, AUSENTE.

RN-04. Cancelar una cita debe registrar motivo, actor y fecha.

RN-05. La aplicación no almacenará diagnóstico médico sensible ni prescripciones; el alcance académico se limita a información administrativa y seguimiento básico.

RN-06. Todo cambio de estado de una cita debe quedar auditado.

RN-07. Un profesional sólo puede visualizar su propia agenda y los pacientes vinculados a sus atenciones; administración tiene acceso ampliado.

RN-08. Los recordatorios deben ser idempotentes para evitar duplicados.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Usuario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Rol	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Paciente	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Profesional	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	Especialidad	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Disponibilidad	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	Cita	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	HistorialEstadoCita	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	NotaSeguimiento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	Notificacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	AuditoriaEvento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de pacientes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de profesionales y especialidades con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de disponibilidad y agenda con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de citas y estados con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de atención/seguimiento administrativo con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de reportes y panel con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de auditoría con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe permitir crear una cita válida desde web y móvil.
RF-14	El sistema debe permitir bloquear doble reserva del profesional.
RF-15	El sistema debe permitir reflejar cambios de estado en tiempo real lógico tras sincronización.
RF-16	El sistema debe permitir persistir historial de cambios.
RF-17	El sistema debe permitir filtrar reportes por rango de fechas y profesional.
RF-18	El sistema debe permitir ejecutar aplicación completa mediante Docker Compose.

H. Requisitos de experiencia web

- Dashboard de ocupación y citas del día
- CRUD de pacientes y profesionales
- Calendario de agenda con filtros
- Gestión de citas y cambios de estado
- Reportes por fecha, profesional, especialidad y estado
- Auditoría de operaciones

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Registro/inicio de sesión
- Consulta de disponibilidad
- Solicitud, confirmación y cancelación de cita
- Mis citas e historial
- Notificaciones y recordatorios
- Vista del profesional: agenda personal y cambio de estado

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Paciente consulta disponibilidad -> solicita cita -> recepción/profesional confirma -> se atiende/cierra -> historial y notificación quedan registrados.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/usuario	GET/POST	Consultar/crear usuario según permisos
/usuario/{id}	GET/PUT/PATCH	Consultar o actualizar usuario
/rol	GET/POST	Consultar/crear rol según permisos
/rol/{id}	GET/PUT/PATCH	Consultar o actualizar rol
/paciente	GET/POST	Consultar/crear paciente según permisos
/paciente/{id}	GET/PUT/PATCH	Consultar o actualizar paciente
/profesional	GET/POST	Consultar/crear profesional según permisos
/profesional/{id}	GET/PUT/PATCH	Consultar o actualizar profesional
/especialidad	GET/POST	Consultar/crear especialidad según permisos
/especialidad/{id}	GET/PUT/PATCH	Consultar o actualizar especialidad
/disponibilidad	GET/POST	Consultar/crear disponibilidad según permisos
/disponibilidad/{id}	GET/PUT/PATCH	Consultar o actualizar disponibilidad
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Crear una cita válida desde web y móvil.
- Bloquear doble reserva del profesional.
- Reflejar cambios de estado en tiempo real lógico tras sincronización.
- Persistir historial de cambios.
- Filtrar reportes por rango de fechas y profesional.
- Ejecutar aplicación completa mediante Docker Compose.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente opcional para redactar mensajes administrativos de recordatorio y resumir indicadores operativos; nunca diagnóstico, prescripción ni recomendaciones médicas.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-01, el equipo debe dejar modelados los conceptos Usuario, Rol, Paciente, Profesional, Especialidad y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: *Paciente consulta disponibilidad -> solicita cita -> recepción/profesional confirma -> se atiende/cierra -> historial y notificación quedan registrados. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.*

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-01, deben estar operativos los roles *Administrador, Recepcionista, Profesional de salud, Paciente con permisos diferenciados* y la mayor parte de los módulos obligatorios: *Identidad y perfiles, Pacientes, Profesionales y especialidades, Disponibilidad y agenda, Citas y estados, Atención/seguimiento administrativo, Notificaciones.*

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.

- Manual técnico de instalación/despliegue.
- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-01, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

02. AutoCare 360 - Taller Automotriz, Órdenes de Servicio y Seguimiento

Campo	Definición
Código	PA-02
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Un taller multimarca necesita abandonar formularios físicos para registrar clientes, vehículos, diagnósticos preliminares, órdenes de servicio, tareas, repuestos estimados, estados y entrega. El cliente quiere seguimiento web interno y una app móvil para que el propietario consulte el avance.

B. Objetivo contractual del producto

Desarrollar una plataforma para controlar el ciclo completo de una orden de servicio desde la recepción del vehículo hasta su entrega, con evidencias, responsables, estimaciones y trazabilidad.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Asesor de servicio	Opera los procesos diarios habilitados para su función y registra información transaccional.
Técnico	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Cliente/propietario	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Clientes
- Vehículos
- Catálogo de servicios
- Órdenes de servicio
- Tareas técnicas
- Estimaciones
- Evidencias
- Estados y entregas
- Notificaciones
- Reportes

E. Reglas de negocio obligatorias

RN-01. Cada vehículo debe asociarse a un propietario activo.

RN-02. Una orden tiene estados: ABIERTA, DIAGNOSTICO, ESPERA_APROBACION, EN_TRABAJO, LISTA_ENTREGA, ENTREGADA, CANCELADA.

RN-03. Sólo una orden activa principal por vehículo, salvo autorización administrativa registrada.

RN-04. Toda estimación debe versionarse cuando cambien costos o alcance.

RN-05. El cliente puede aprobar o rechazar una estimación, quedando registro de fecha y decisión.

RN-06. Las tareas deben tener responsable y estado.

RN-07. La entrega requiere cerrar tareas obligatorias y registrar fecha, kilometraje de salida y observaciones.

RN-08. Las evidencias adjuntas deben asociarse a la orden y tener descripción.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Cliente	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Vehiculo	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	ServicioCatalogo	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	OrdenServicio	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	TareaOrden	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Estimacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	ItemEstimacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Evidencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	HistorialEstadoOrden	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	AprobacionCliente	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	Notificacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de clientes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de vehículos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de catálogo de servicios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de órdenes de servicio con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de tareas técnicas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de estimaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de evidencias con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de estados y entregas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir registrar cliente y vehículo sin duplicar placa/VIN configurado.
RF-16	El sistema debe permitir abrir orden y moverla por el flujo válido.
RF-17	El sistema debe permitir aprobar estimación desde móvil.
RF-18	El sistema debe permitir asignar y cerrar tareas.
RF-19	El sistema debe permitir adjuntar evidencias.
RF-20	El sistema debe permitir consultar historial completo de la orden.

H. Requisitos de experiencia web

- Recepción de vehículo
- Kanban/tabla de órdenes por estado
- Asignación de técnicos y tareas
- Estimación de servicios/repuestos

- Carga de evidencias
- Cierre y entrega
- Indicadores de tiempos y carga de trabajo

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Mis vehículos
- Estado de órdenes
- Detalle de trabajos y evidencias
- Aprobar/rechazar estimación
- Notificaciones
- Vista técnico: mis tareas, evidencias y avance

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Asesor abre orden -> técnico recibe tareas -> se genera estimación -> cliente aprueba desde móvil -> técnico actualiza trabajo -> asesor entrega vehículo.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/cliente	GET/POST	Consultar/crear cliente según permisos
/cliente/{id}	GET/PUT/PATCH	Consultar o actualizar cliente
/vehiculo	GET/POST	Consultar/crear vehiculo según permisos
/vehiculo/{id}	GET/PUT/PATCH	Consultar o actualizar vehiculo
/serviciocatalogo	GET/POST	Consultar/crear serviciocatalogo según permisos
/serviciocatalogo/{id}	GET/PUT/PATCH	Consultar o actualizar serviciocatalogo
/ordenservicio	GET/POST	Consultar/crear ordenservicio según permisos
/ordenservicio/{id}	GET/PUT/PATCH	Consultar o actualizar ordenservicio
/tareaorden	GET/POST	Consultar/crear tareaorden según permisos
/tareaorden/{id}	GET/PUT/PATCH	Consultar o actualizar tareaorden
/estimacion	GET/POST	Consultar/crear estimacion según permisos
/estimacion/{id}	GET/PUT/PATCH	Consultar o actualizar estimacion
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Registrar cliente y vehículo sin duplicar placa/VIN configurado.
- Abrir orden y moverla por el flujo válido.
- Aprobar estimación desde móvil.
- Asignar y cerrar tareas.
- Adjuntar evidencias.
- Consultar historial completo de la orden.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para convertir observaciones textuales del asesor en una lista estructurada de tareas sugeridas; debe requerir confirmación humana antes de persistir.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-02, el equipo debe dejar modelados los conceptos Cliente, Vehículo, ServicioCatálogo, OrdenServicio, TareaOrden y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Asesor abre orden -> técnico recibe tareas -> se genera estimación -> cliente aprueba desde móvil -> técnico actualiza trabajo -> asesor entrega vehículo. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-02, deben estar operativos los roles Administrador, Asesor de servicio, Técnico, Cliente/propietario con permisos diferenciados y la mayor parte de los módulos obligatorios: Clientes, Vehículos, Catálogo de servicios, Órdenes de servicio, Tareas técnicas, Estimaciones, Evidencias.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.
- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-02, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

03. GastroReserva - Reservas, Mesas, Pedidos y Experiencia de Restaurante

Campo	Definición
Código	PA-03
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Un restaurante de tamaño medio requiere centralizar reservas, disponibilidad de mesas, atención por turnos, pedidos simples y seguimiento del estado de consumo. Hoy mezcla llamadas, papel y mensajería. El cliente exige experiencia web para administración y móvil para clientes y personal de salón.

B. Objetivo contractual del producto

Crear un sistema que gestione reservas y operación básica de salón, permitiendo controlar capacidad, mesas, clientes, pedidos, estados y retroalimentación sin convertirse en un sistema contable.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Host/recepción	Opera los procesos diarios habilitados para su función y registra información transaccional.
Mesero	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Cliente	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Clientes
- Mesas y zonas
- Turnos
- Reservas
- Check-in
- Carta básica
- Pedidos
- Estados de atención
- Feedback
- Panel y reportes

E. Reglas de negocio obligatorias

RN-01. Una reserva debe respetar capacidad de mesa y turno.

RN-02. No se permiten reservas solapadas de una misma mesa en el mismo intervalo.

RN-03. Estados de reserva: SOLICITADA, CONFIRMADA, SENTADA, FINALIZADA, CANCELADA, NO_SHOW.

RN-04. El pedido sólo puede crearse para una reserva sentada o una mesa abierta.

RN-05. Los ítems del pedido conservan precio histórico al momento de agregarse.

RN-06. No se implementará contabilidad ni facturación fiscal; sólo total operativo del consumo.

RN-07. El feedback sólo puede registrarse tras finalizar la visita.

RN-08. Toda reasignación de mesa debe quedar trazada.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Cliente	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Zona	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Mesa	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Turno	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	Reserva	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	HistorialReserva	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	ProductoMenu	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Pedido	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	ItemPedido	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	EstadoPedido	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	Feedback	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de clientes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de mesas y zonas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de turnos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de reservas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de check-in con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de carta básica con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de pedidos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de estados de atención con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de feedback con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de panel y reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir reservar sólo si existe capacidad.
RF-16	El sistema debe permitir evitar solapamientos.
RF-17	El sistema debe permitir realizar check-in y asignación.
RF-18	El sistema debe permitir crear pedido con totales correctos.
RF-19	El sistema debe permitir actualizar estado desde móvil de mesero.
RF-20	El sistema debe permitir generar indicadores de ocupación y no-show.

H. Requisitos de experiencia web

- Mapa/listado de mesas
- Agenda de reservas
- Gestión de turnos y capacidad
- Check-in y asignación de mesa
- Carta y disponibilidad
- Pedidos y estados

- Panel de ocupación y no-show

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Buscar horario
- Crear/cancelar reserva
- Mis reservas
- Notificaciones
- Feedback
- Vista mesero: mesas asignadas, pedido y cambio de estado

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Cliente consulta horario -> reserva -> host realiza check-in -> mesero gestiona pedido -> se finaliza visita -> cliente registra feedback.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/cliente	GET/POST	Consultar/crear cliente según permisos
/cliente/{id}	GET/PUT/PATCH	Consultar o actualizar cliente
/zona	GET/POST	Consultar/crear zona según permisos
/zona/{id}	GET/PUT/PATCH	Consultar o actualizar zona
/mesa	GET/POST	Consultar/crear mesa según permisos
/mesa/{id}	GET/PUT/PATCH	Consultar o actualizar mesa
/turno	GET/POST	Consultar/crear turno según permisos
/turno/{id}	GET/PUT/PATCH	Consultar o actualizar turno
/reserva	GET/POST	Consultar/crear reserva según permisos
/reserva/{id}	GET/PUT/PATCH	Consultar o actualizar reserva
/historialreserva	GET/POST	Consultar/crear historialreserva según permisos
/historialreserva/{id}	GET/PUT/PATCH	Consultar o actualizar historialreserva
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Reservar sólo si existe capacidad.
- Evitar solapamientos.
- Realizar check-in y asignación.
- Crear pedido con totales correctos.
- Actualizar estado desde móvil de mesero.
- Generar indicadores de ocupación y no-show.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para clasificar comentarios de clientes por tema y sentimiento operativo, con resumen para el administrador.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-03, el equipo debe dejar modelados los conceptos Cliente, Zona, Mesa, Turno, Reserva y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Cliente consulta horario -> reserva -> host realiza check-in -> mesero gestiona pedido -> se finaliza visita -> cliente registra feedback. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-03, deben estar operativos los roles Administrador, Host/recepción, Mesero, Cliente con permisos diferenciados y la mayor parte de los módulos obligatorios: Clientes, Mesas y zonas, Turnos, Reservas, Check-in, Carta básica, Pedidos.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-03, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

04. CondoFlow - Administración Operativa de Condominio

Campo	Definición
Código	PA-04
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Un condominio residencial necesita gestionar unidades, propietarios/inquilinos, avisos, incidencias, reservas de áreas comunes, visitas y seguimiento de tareas del personal. El proyecto no reemplaza sistemas contables ni de cobranza formal.

B. Objetivo contractual del producto

Construir una plataforma de gestión comunitaria y operativa que facilite comunicación, reservas, incidencias, control de visitas y trazabilidad de atención.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Portería/seguridad	Opera los procesos diarios habilitados para su función y registra información transaccional.
Residente	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.
Personal de mantenimiento	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.

D. Alcance funcional cerrado

- Unidades y residentes
- Comunicados
- Incidencias
- Áreas comunes
- Reservas
- Visitas
- Tareas de mantenimiento
- Notificaciones
- Panel operativo

E. Reglas de negocio obligatorias

RN-01. Una unidad puede tener varios residentes con vigencia temporal.

RN-02. Las reservas de áreas comunes deben respetar aforo, horario y no solapamiento.

RN-03. Las incidencias siguen estados: REPORTADA, ASIGNADA, EN_PROCESO, RESUELTA, CERRADA, RECHAZADA.

RN-04. Cada incidencia debe registrar categoría, prioridad, descripción, unidad/área y responsable cuando corresponda.

RN-05. Una visita debe vincularse a una unidad y tener fecha prevista.

RN-06. Portería sólo puede operar visitas y consultas autorizadas.

RN-07. Los comunicados pueden dirigirse a todos o a segmentos.

RN-08. No se implementará contabilidad, expensas ni pagos reales.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Unidad	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Persona	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Residencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	AreaComun	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	ReservaArea	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Incidencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	CategoriaIncidencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	TareaMantenimiento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	Visita	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	Comunicado	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	Notificacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de unidades y residentes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de comunicados con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de incidencias con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de áreas comunes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de reservas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de visitas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de tareas de mantenimiento con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de panel operativo con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe permitir registrar residencia vigente.
RF-15	El sistema debe permitir evitar solapamiento de reservas.
RF-16	El sistema debe permitir reportar incidencia desde móvil con evidencia.
RF-17	El sistema debe permitir asignar y resolver incidencia.
RF-18	El sistema debe permitir registrar visita y validar consulta de portería.
RF-19	El sistema debe permitir filtrar indicadores por categoría/estado.

H. Requisitos de experiencia web

- Administración de unidades/residentes
- Tablero de incidencias
- Calendario de áreas comunes
- Gestión de visitas
- Comunicados
- Tareas de mantenimiento
- Indicadores de tiempos de resolución

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Mis comunicados
- Reportar incidencia con evidencia
- Seguimiento de incidencia
- Reservar área común
- Registrar visita esperada
- Notificaciones
- Mantenimiento: mis tareas

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Residente reporta incidencia -> administración clasifica/asigna -> mantenimiento trabaja -> residente consulta avance -> incidencia se resuelve y cierra.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/unidad	GET/POST	Consultar/crear unidad según permisos
/unidad/{id}	GET/PUT/PATCH	Consultar o actualizar unidad
/persona	GET/POST	Consultar/crear persona según permisos
/persona/{id}	GET/PUT/PATCH	Consultar o actualizar persona
/residencia	GET/POST	Consultar/crear residencia según permisos
/residencia/{id}	GET/PUT/PATCH	Consultar o actualizar residencia
/areacomun	GET/POST	Consultar/crear areacomun según permisos
/areacomun/{id}	GET/PUT/PATCH	Consultar o actualizar areacomun
/reservaarea	GET/POST	Consultar/crear reservaarea según permisos
/reservaarea/{id}	GET/PUT/PATCH	Consultar o actualizar reservaarea
/incidencia	GET/POST	Consultar/crear incidencia según permisos
/incidencia/{id}	GET/PUT/PATCH	Consultar o actualizar incidencia
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Registrar residencia vigente.
- Evitar solapamiento de reservas.
- Reportar incidencia desde móvil con evidencia.
- Asignar y resolver incidencia.
- Registrar visita y validar consulta de portería.
- Filtrar indicadores por categoría/estado.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para clasificar incidencias por categoría/prioridad sugerida y redactar resumen semanal, siempre con confirmación humana.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-04, el equipo debe dejar modelados los conceptos Unidad, Persona, Residencia, AreaComun, ReservaArea y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Residente reporta incidencia -> administración clasifica/asigna -> mantenimiento trabaja -> residente consulta avance -> incidencia se resuelve y cierra. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-04, deben estar operativos los roles Administrador, Portería/seguridad, Residente, Personal de mantenimiento con permisos diferenciados y la mayor parte de los módulos obligatorios: Unidades y residentes, Comunicados, Incidencias, Áreas comunes, Reservas, Visitas, Tareas de mantenimiento.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-04, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

05. EduMentor - Tutorías Académicas, Sesiones y Seguimiento de Aprendizaje

Campo	Definición
Código	PA-05
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Un centro de apoyo académico coordina tutores, estudiantes, materias, disponibilidad, reservas de sesiones y seguimiento de objetivos. Actualmente todo se gestiona por mensajes y hojas de cálculo.

B. Objetivo contractual del producto

Desarrollar una plataforma para organizar tutorías, disponibilidad, reservas, objetivos de aprendizaje, evidencias y retroalimentación, diferenciando claramente información académica de calificaciones oficiales.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Tutor	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Estudiante	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Perfiles
- Materias
- Competencias
- Disponibilidad
- Reservas
- Sesiones
- Objetivos
- Recursos
- Feedback
- Reportes

E. Reglas de negocio obligatorias

RN-01. Un tutor sólo puede ofrecer materias habilitadas en su perfil.

RN-02. No puede existir solapamiento de sesiones para un tutor o estudiante.

RN-03. Estados: SOLICITADA, CONFIRMADA, REALIZADA, CANCELADA, AUSENTE.

RN-04. Cada sesión realizada debe tener un resumen de temas y próximos pasos.

RN-05. Los objetivos pertenecen al estudiante y pueden vincularse a sesiones.

RN-06. Los recursos compartidos deben tener autor y contexto.

RN-07. El sistema no sustituye el sistema académico oficial ni registra notas oficiales.

RN-08. El feedback se habilita después de una sesión realizada.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Usuario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Tutor	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Estudiante	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Materia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	TutorMateria	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Disponibilidad	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	SesionTutoría	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	ObjetivoAprendizaje	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	Recurso	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	Feedback	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	HistorialSesion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.

RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de perfiles con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de materias con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de competencias con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de disponibilidad con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de reservas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de sesiones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de objetivos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de recursos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de feedback con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir buscar tutores por materia.
RF-16	El sistema debe permitir reservar sin solapamiento.
RF-17	El sistema debe permitir cerrar sesión con resumen.
RF-18	El sistema debe permitir crear/actualizar objetivos.
RF-19	El sistema debe permitir compartir recurso.
RF-20	El sistema debe permitir visualizar avance de objetivos e historial.

H. Requisitos de experiencia web

- Gestión de tutores y materias
- Calendario de disponibilidad
- Reservas y estados
- Registro de sesiones
- Objetivos y recursos
- Panel de demanda y cumplimiento

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Buscar tutor/materia
- Reservar/cancelar sesión
- Mis sesiones
- Mis objetivos
- Recursos compartidos
- Feedback
- Tutor: agenda y cierre de sesión

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Estudiante busca tutor -> reserva sesión -> tutor confirma -> realiza sesión y registra resumen -> estudiante revisa próximos pasos y feedback.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/usuario	GET/POST	Consultar/crear usuario según permisos
/usuario/{id}	GET/PUT/PATCH	Consultar o actualizar usuario
/tutor	GET/POST	Consultar/crear tutor según permisos
/tutor/{id}	GET/PUT/PATCH	Consultar o actualizar tutor
/estudiante	GET/POST	Consultar/crear estudiante según permisos
/estudiante/{id}	GET/PUT/PATCH	Consultar o actualizar estudiante
/materia	GET/POST	Consultar/crear materia según permisos
/materia/{id}	GET/PUT/PATCH	Consultar o actualizar materia
/tutormateria	GET/POST	Consultar/crear tutormateria según permisos
/tutormateria/{id}	GET/PUT/PATCH	Consultar o actualizar tutormateria
/disponibilidad	GET/POST	Consultar/crear disponibilidad según permisos
/disponibilidad/{id}	GET/PUT/PATCH	Consultar o actualizar disponibilidad
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Buscar tutores por materia.
- Reservar sin solapamiento.
- Cerrar sesión con resumen.
- Crear/actualizar objetivos.
- Compartir recurso.
- Visualizar avance de objetivos e historial.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente que, a partir del resumen de una tutoría, propone una lista de próximos pasos o preguntas de repaso; no genera calificación académica automática.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-05, el equipo debe dejar modelados los conceptos Usuario, Tutor, Estudiante, Materia, TutorMateria y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Estudiante busca tutor -> reserva sesión -> tutor confirma -> realiza sesión y registra resumen -> estudiante revisa próximos pasos y feedback. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-05, deben estar operativos los roles Administrador, Tutor, Estudiante con permisos diferenciados y la mayor parte de los módulos obligatorios: Perfiles, Materias, Competencias, Disponibilidad, Reservas, Sesiones, Objetivos.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-05, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

06. StockFlow - Inventario, Compras Internas y Movimientos de Almacén

Campo	Definición
Código	PA-06
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Una pyme con un almacén central y dos depósitos necesita controlar productos, categorías, existencias, entradas, salidas, transferencias y solicitudes internas. El sistema actual no ofrece trazabilidad ni alertas confiables.

B. Objetivo contractual del producto

Construir un sistema de inventario transaccional con movimientos auditables, stock por ubicación, solicitudes y aprobaciones, evitando implementar contabilidad o facturación.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Encargado de almacén	Opera los procesos diarios habilitados para su función y registra información transaccional.
Solicitante interno	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.
Supervisor	Supervisa, aprueba o coordina procesos específicos, con visibilidad ampliada y trazabilidad.

D. Alcance funcional cerrado

- Productos
- Categorías
- Ubicaciones
- Stock
- Movimientos
- Solicitudes internas
- Transferencias
- Aprobaciones
- Alertas
- Reportes

E. Reglas de negocio obligatorias

RN-01. El stock no se edita directamente; cambia únicamente mediante movimientos válidos.

RN-02. Todo movimiento debe tener tipo, cantidad positiva, fecha, usuario y referencia.

RN-03. No se permite stock negativo salvo configuración explícita del proyecto, que por defecto estará deshabilitada.

RN-04. Una transferencia genera salida en origen y entrada en destino dentro de una misma transacción lógica.

RN-05. Las solicitudes siguen estados: BORRADOR, ENVIADA, APROBADA, RECHAZADA, ATENDIDA, CANCELADA.

RN-06. Los productos deben tener código único.

RN-07. Las alertas de stock mínimo se calculan por producto/ubicación.

RN-08. No se implementará contabilidad, cuentas por pagar ni facturación fiscal.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Producto	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Categoria	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Ubicacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Stock	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	MovimientoInventario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Solicitud	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	ItemSolicitud	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Aprobacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	Transferencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	AlertaStock	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de productos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de categorías con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de ubicaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de stock con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de movimientos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de solicitudes internas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de transferencias con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de aprobaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de alertas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir impedir stock negativo.
RF-16	El sistema debe permitir registrar movimiento y recalcular stock de forma transaccional.
RF-17	El sistema debe permitir transferir entre ubicaciones sin inconsistencias.
RF-18	El sistema debe permitir aprobar solicitud y atenderla.
RF-19	El sistema debe permitir consultar kardex.
RF-20	El sistema debe permitir mostrar alertas de mínimos.

H. Requisitos de experiencia web

- CRUD de productos/ubicaciones
- Kardex simplificado
- Registrar entradas/salidas
- Solicitudes y aprobación
- Transferencias
- Dashboard de stock bajo

- Reportes de movimientos

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Consulta de stock
- Crear solicitud interna
- Aprobar/rechazar solicitud
- Registrar movimiento autorizado
- Escaneo opcional de código manual/QR académico
- Notificaciones de stock bajo

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Solicitante crea solicitud -> supervisor aprueba -> almacén atiende mediante movimiento -> stock se actualiza -> kardex y alerta reflejan el resultado.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/producto	GET/POST	Consultar/crear producto según permisos
/producto/{id}	GET/PUT/PATCH	Consultar o actualizar producto
/categoria	GET/POST	Consultar/crear categoria según permisos
/categoria/{id}	GET/PUT/PATCH	Consultar o actualizar categoria
/ubicacion	GET/POST	Consultar/crear ubicacion según permisos
/ubicacion/{id}	GET/PUT/PATCH	Consultar o actualizar ubicacion
/stock	GET/POST	Consultar/crear stock según permisos
/stock/{id}	GET/PUT/PATCH	Consultar o actualizar stock
/movimientoinventario	GET/POST	Consultar/crear movimientoinventario según permisos
/movimientoinventario/{id}	GET/PUT/PATCH	Consultar o actualizar movimientoinventario
/solicitud	GET/POST	Consultar/crear solicitud según permisos
/solicitud/{id}	GET/PUT/PATCH	Consultar o actualizar solicitud
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Impedir stock negativo.
- Registrar movimiento y recalcular stock de forma transaccional.
- Transferir entre ubicaciones sin inconsistencias.
- Aprobar solicitud y atenderla.
- Consultar kardex.
- Mostrar alertas de mínimos.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente opcional para resumir tendencias de movimientos y explicar anomalías simples a partir de datos agregados; no realiza decisiones autónomas de compra.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-06, el equipo debe dejar modelados los conceptos *Producto, Categoría, Ubicación, Stock, MovimientoInventario* y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Solicitante crea solicitud -> supervisor aprueba -> almacén atiende mediante movimiento -> stock se actualiza -> kardex y alerta reflejan el resultado. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-06, deben estar operativos los roles Administrador, Encargado de almacén, Solicitante interno, Supervisor con permisos diferenciados y la mayor parte de los módulos obligatorios: Productos, Categorías, Ubicaciones, Stock, Movimientos, Solicitudes internas, Transferencias.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-06, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

07. FixDesk - Mesa de Ayuda y Gestión de Soporte Técnico

Campo	Definición
Código	PA-07
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Una organización necesita centralizar solicitudes de soporte de hardware, software, redes y acceso. Actualmente los pedidos llegan por distintos canales, no se conoce responsable ni tiempo de resolución y no existe base de conocimiento.

B. Objetivo contractual del producto

Implementar una mesa de ayuda con tickets, categorías, prioridades, asignación, comentarios, evidencias, SLA académico, base de conocimiento y métricas de atención.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Agente de soporte	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Solicitante	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.
Supervisor	Supervisa, aprueba o coordina procesos específicos, con visibilidad ampliada y trazabilidad.

D. Alcance funcional cerrado

- Usuarios/áreas
- Catálogo de categorías
- Tickets
- Asignaciones
- Comentarios
- Adjuntos
- SLA
- Base de conocimiento
- Notificaciones
- Reportes

E. Reglas de negocio obligatorias

RN-01. Todo ticket debe tener solicitante, categoría, prioridad, asunto y descripción.

RN-02. Estados: NUEVO, ASIGNADO, EN_PROGRESO, ESPERA_USUARIO, RESUELTO, CERRADO, CANCELADO.

RN-03. Cerrar un ticket requiere resolución registrada.

RN-04. La prioridad puede recalcularse con justificación y trazabilidad.

RN-05. El SLA se calcula en función de prioridad y fechas hábiles simplificadas definidas por el proyecto.

RN-06. Los comentarios internos no son visibles al solicitante.

RN-07. Los artículos de conocimiento tienen estado BORRADOR/PUBLICADO.

RN-08. Cada reasignación debe registrarse.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Area	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Usuario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	CategoriaTicket	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Ticket	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	AsignacionTicket	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Comentario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	Adjunto	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	SlaPolitica	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	HistorialTicket	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	ArticuloConocimiento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	Notificacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de catálogo de categorías con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de tickets con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de asignaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de comentarios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de adjuntos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de sla con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de base de conocimiento con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe permitir crear ticket web/móvil.
RF-15	El sistema debe permitir asignar y cambiar estado respetando transiciones.
RF-16	El sistema debe permitir distinguir comentario interno/público.
RF-17	El sistema debe permitir calcular fecha objetivo de SLA.
RF-18	El sistema debe permitir buscar artículos.
RF-19	El sistema debe permitir generar métricas por categoría/agente/estado.

H. Requisitos de experiencia web

- Bandeja de tickets
- Filtros y búsqueda
- Asignación y priorización
- Comentarios/adjuntos
- SLA y vencimientos
- Base de conocimiento
- Dashboard de tiempos y backlog

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Crear ticket con evidencia
- Mis tickets
- Responder comentarios públicos
- Notificaciones
- Agente: tickets asignados, cambio de estado y comentario

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Usuario crea ticket -> sistema/mesa clasifica -> agente recibe -> conversa y trabaja -> registra resolución -> solicitante verifica/cierra.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/area	GET/POST	Consultar/crear area según permisos
/area/{id}	GET/PUT/PATCH	Consultar o actualizar area
/usuario	GET/POST	Consultar/crear usuario según permisos
/usuario/{id}	GET/PUT/PATCH	Consultar o actualizar usuario
/categoriaticket	GET/POST	Consultar/crear categoriaticket según permisos
/categoriaticket/{id}	GET/PUT/PATCH	Consultar o actualizar categoriaticket
/ticket	GET/POST	Consultar/crear ticket según permisos
/ticket/{id}	GET/PUT/PATCH	Consultar o actualizar ticket
/asignacionticket	GET/POST	Consultar/crear asignacionticket según permisos
/asignacionticket/{id}	GET/PUT/PATCH	Consultar o actualizar asignacionticket
/comentario	GET/POST	Consultar/crear comentario según permisos
/comentario/{id}	GET/PUT/PATCH	Consultar o actualizar comentario
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Crear ticket web/móvil.

- Asignar y cambiar estado respetando transiciones.
- Distinguir comentario interno/público.
- Calcular fecha objetivo de SLA.
- Buscar artículos.
- Generar métricas por categoría/agente/estado.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para sugerir categoría, prioridad y artículos de conocimiento relevantes; la decisión final corresponde al agente.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-07, el equipo debe dejar modelados los conceptos *Area*, *Usuario*, *CategoriaTicket*, *Ticket*, *AsignacionTicket* y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Usuario crea ticket -> sistema/mesa clasifica -> agente recibe -> conversa y trabaja -> registra resolución -> solicitante verifica/cierra. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-07, deben estar operativos los roles Administrador, Agente de soporte, Solicitante, Supervisor con permisos diferenciados y la mayor parte de los módulos obligatorios: Usuarios/áreas, Catálogo de categorías, Tickets, Asignaciones, Comentarios, Adjuntos, SLA.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-07, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

08. FitClub - Membresías, Clases y Reservas de Centro Deportivo

Campo	Definición
Código	PA-08
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Un centro deportivo ofrece membresías, clases grupales y sesiones con instructores. Requiere controlar planes, vigencia, cupos, reservas, asistencia y comunicación con socios, sin implementar cobros bancarios reales.

B. Objetivo contractual del producto

Construir una solución para administrar socios, planes, clases, instructores, reservas, asistencia y métricas de ocupación con experiencia móvil para socios.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Recepción	Opera los procesos diarios habilitados para su función y registra información transaccional.
Instructor	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Socio	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Socios
- Planes
- Membresías
- Instructores
- Clases
- Horarios
- Reservas
- Asistencia
- Notificaciones
- Reportes

E. Reglas de negocio obligatorias

RN-01. Una membresía tiene fecha de inicio/fin y estado.

RN-02. Sólo socios con membresía vigente pueden reservar clases restringidas.

RN-03. Cada clase tiene cupo máximo y no admite sobre-reserva.

RN-04. No se permiten reservas duplicadas del mismo socio para una clase.

RN-05. La asistencia sólo puede registrarse sobre reservas válidas o ingreso autorizado.

RN-06. Cancelaciones tardías pueden marcarse para fines académicos, sin cobro real.

RN-07. Un instructor no puede tener clases solapadas.

RN-08. No se procesarán pagos reales ni información bancaria.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Socio	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Plan	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Membresia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Instructor	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	Clase	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	HorarioClase	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	ReservaClase	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Asistencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	HistorialMembresia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	Notificacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.

RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de socios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de planes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de membresías con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de instructores con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de clases con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de horarios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de reservas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de asistencia con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir validar vigencia antes de reservar.
RF-16	El sistema debe permitir respetar cupo.
RF-17	El sistema debe permitir evitar solapamiento de instructor.
RF-18	El sistema debe permitir registrar asistencia.
RF-19	El sistema debe permitir mostrar historial de reservas.
RF-20	El sistema debe permitir reportar ocupación por clase/horario.

H. Requisitos de experiencia web

- Socios/membresías
- Programación de clases
- Calendario de instructores
- Reservas y lista de participantes
- Control de asistencia
- Panel de ocupación y vigencias

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Mi membresía
- Explorar clases
- Reservar/cancelar
- Mis reservas
- Check-in académico mediante código
- Notificaciones
- Instructor: lista y asistencia

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Socio con membresía vigente consulta clase -> reserva -> instructor ve lista -> registra asistencia -> dashboard actualiza ocupación.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/socio	GET/POST	Consultar/crear socio según permisos
/socio/{id}	GET/PUT/PATCH	Consultar o actualizar socio
/plan	GET/POST	Consultar/crear plan según permisos
/plan/{id}	GET/PUT/PATCH	Consultar o actualizar plan
/membresia	GET/POST	Consultar/crear membresia según permisos
/membresia/{id}	GET/PUT/PATCH	Consultar o actualizar membresia
/instructor	GET/POST	Consultar/crear instructor según permisos
/instructor/{id}	GET/PUT/PATCH	Consultar o actualizar instructor
/clase	GET/POST	Consultar/crear clase según permisos
/clase/{id}	GET/PUT/PATCH	Consultar o actualizar clase
/horarioclase	GET/POST	Consultar/crear horarioclase según permisos
/horarioclase/{id}	GET/PUT/PATCH	Consultar o actualizar horarioclase
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Validar vigencia antes de reservar.
- Respetar cupo.

- Evitar solapamiento de instructor.
- Registrar asistencia.
- Mostrar historial de reservas.
- Reportar ocupación por clase/horario.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para generar resúmenes operativos de ocupación y sugerir horarios con baja/alta demanda a partir de datos históricos simples.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-08, el equipo debe dejar modelados los conceptos Socio, Plan, Membresía, Instructor, Clase y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.

- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Socio con membresía vigente consulta clase -> reserva -> instructor ve lista -> registra asistencia -> dashboard actualiza ocupación. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-08, deben estar operativos los roles Administrador, Recepción, Instructor, Socio con permisos diferenciados y la mayor parte de los módulos obligatorios: Socios, Planes, Membresías, Instructores, Clases, Horarios, Reservas.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.
- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.

- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-08, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

09. PetCare Manager - Clínica Veterinaria y Seguimiento de Mascotas

Campo	Definición
Código	PA-09
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Una clínica veterinaria necesita gestionar propietarios, mascotas, agenda, motivos de consulta, controles administrativos, vacunaciones registradas, recordatorios y seguimiento. Actualmente la información está distribuida entre papel y mensajes.

B. Objetivo contractual del producto

Desarrollar una plataforma de gestión veterinaria académica centrada en agenda, historial básico de atención, vacunaciones y recordatorios, con controles de acceso y aplicación móvil para propietarios.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Recepción	Opera los procesos diarios habilitados para su función y registra información transaccional.
Veterinario	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Propietario	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Propietarios
- Mascotas
- Especies/razas
- Agenda
- Atenciones
- Vacunaciones
- Recordatorios
- Archivos
- Notificaciones
- Reportes

E. Reglas de negocio obligatorias

RN-01. Cada mascota debe tener al menos un propietario responsable.

RN-02. No se permiten citas solapadas para el mismo veterinario.

RN-03. Estados de cita equivalentes al flujo definido.

RN-04. Una vacunación debe registrar vacuna, fecha aplicada y próxima fecha opcional.

RN-05. Los recordatorios se generan desde fechas próximas, evitando duplicados.

RN-06. El historial visible al propietario sólo incluye información autorizada por el alcance académico.

RN-07. Los archivos deben estar asociados a mascota o atención.

RN-08. Toda corrección de una atención cerrada debe quedar auditada.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Propietario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	Mascota	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Especie	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Raza	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	Veterinario	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Cita	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	Atencion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Vacunacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	Recordatorio	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	ArchivoMascota	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	HistorialCita	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de propietarios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de mascotas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de especies/razas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de agenda con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de atenciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de vacunaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de recordatorios con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de archivos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de notificaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir registrar mascota con propietario.
RF-16	El sistema debe permitir agendar sin solapamiento.
RF-17	El sistema debe permitir registrar atención y vacuna.
RF-18	El sistema debe permitir generar recordatorio único.
RF-19	El sistema debe permitir consultar historial desde móvil.
RF-20	El sistema debe permitir filtrar próximos controles.

H. Requisitos de experiencia web

- Registro de propietarios/mascotas
- Agenda veterinaria
- Atención y seguimiento
- Vacunaciones
- Recordatorios
- Archivos

- Indicadores de próximas vacunas/citas

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Mis mascotas
- Reservar/cancelar cita
- Historial básico
- Vacunaciones y próximas fechas
- Notificaciones
- Veterinario: agenda y registro de atención

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Propietario registra/consulta mascota -> solicita cita -> recepción confirma -> veterinario registra atención/vacunación -> propietario recibe recordatorio.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/propietario	GET/POST	Consultar/crear propietario según permisos
/propietario/{id}	GET/PUT/PATCH	Consultar o actualizar propietario
/mascota	GET/POST	Consultar/crear mascota según permisos
/mascota/{id}	GET/PUT/PATCH	Consultar o actualizar mascota
/especie	GET/POST	Consultar/crear especie según permisos
/especie/{id}	GET/PUT/PATCH	Consultar o actualizar especie
/raza	GET/POST	Consultar/crear raza según permisos
/raza/{id}	GET/PUT/PATCH	Consultar o actualizar raza
/veterinario	GET/POST	Consultar/crear veterinario según permisos
/veterinario/{id}	GET/PUT/PATCH	Consultar o actualizar veterinario
/cita	GET/POST	Consultar/crear cita según permisos
/cita/{id}	GET/PUT/PATCH	Consultar o actualizar cita
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Registrar mascota con propietario.
- Agendar sin solapamiento.
- Registrar atención y vacuna.
- Generar recordatorio único.
- Consultar historial desde móvil.
- Filtrar próximos controles.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente limitado a redactar recordatorios y resumir historial administrativo; no diagnostica ni prescribe tratamientos.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-09, el equipo debe dejar modelados los conceptos *Propietario, Mascota, Especie, Raza, Veterinario* y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Propietario registra/consulta mascota -> solicita cita -> recepción confirma -> veterinario registra atención/vacunación -> propietario recibe recordatorio. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-09, deben estar operativos los roles Administrador, Recepción, Veterinario, Propietario con permisos diferenciados y la mayor parte de los módulos obligatorios: Propietarios, Mascotas, Especies/razas, Agenda, Atenciones, Vacunaciones, Recordatorios.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-09, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

10. EventHub - Gestión de Eventos, Inscripciones y Control de Asistencia

Campo	Definición
Código	PA-10
Tipo	Proyecto integrador full-stack web + móvil
Complejidad	Intermedia controlada; equivalente al resto del banco
Stack obligatorio	Java 21, Spring Boot, PostgreSQL, React + TypeScript, React Native + TypeScript, Docker, GitHub Actions

A. Situación del cliente

Una institución organiza talleres, charlas y actividades presenciales/híbridas. Necesita publicar eventos, controlar cupos, inscripciones, listas de espera, asistencia, certificados académicos simples y comunicación con participantes.

B. Objetivo contractual del producto

Construir una plataforma para administrar el ciclo de vida de eventos e inscripciones, desde publicación hasta asistencia y cierre, con panel web y app móvil.

C. Actores y responsabilidades

Actor	Responsabilidad principal
Administrador	Configura catálogos, usuarios, parámetros y supervisa la operación completa.
Organizador	Supervisa, aprueba o coordina procesos específicos, con visibilidad ampliada y trazabilidad.
Facilitador	Ejecuta el trabajo especializado asignado, actualiza estados y aporta evidencias/seguimiento.
Participante	Usuario externo o beneficiario que consulta, solicita, confirma, cancela o realiza acciones de autoservicio autorizadas.

D. Alcance funcional cerrado

- Eventos
- Sesiones
- Sedes/salas
- Inscripciones
- Lista de espera
- Asistencia
- Comunicaciones
- Certificados académicos
- Feedback
- Reportes

E. Reglas de negocio obligatorias

RN-01. Un evento debe tener responsable, modalidad, fechas, capacidad y estado.

RN-02. Estados: BORRADOR, PUBLICADO, EN_CURSO, FINALIZADO, CANCELADO.

RN-03. La inscripción debe respetar cupo; si se llena, pasa a lista de espera cuando esté habilitada.

RN-04. No puede existir inscripción duplicada por participante/evento.

RN-05. La promoción desde lista de espera debe ser ordenada y trazable.

RN-06. La asistencia se registra por sesión o evento según configuración.

RN-07. El certificado académico simple sólo se habilita si se cumple el porcentaje mínimo de asistencia.

RN-08. No se implementarán pagos reales ni emisión legal de certificados.

F. Modelo de información mínimo esperado

El equipo debe diseñar el modelo relacional definitivo. La siguiente lista representa entidades o conceptos mínimos que deben aparecer en el análisis; pueden surgir tablas adicionales para relaciones N:M, históricos, auditoría, catálogos o normalización.

#	Entidad / concepto	Responsabilidad de datos
1	Evento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
2	SesionEvento	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
3	Sede	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
4	Sala	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
5	Participante	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
6	Inscripcion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
7	ListaEspera	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
8	Asistencia	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
9	Comunicacion	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
10	Certificado	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.
11	Feedback	Debe tener identidad, atributos, restricciones, relaciones y justificación de persistencia coherentes con el dominio.

G. Requisitos funcionales predefinidos

ID	Requisito funcional
----	---------------------

RF-01	El sistema debe autenticar usuarios y resolver permisos de acuerdo con el rol vigente.
RF-02	El sistema debe permitir consultar listados con filtros relevantes al dominio y devolver mensajes claros cuando no existan resultados.
RF-03	El sistema debe validar datos obligatorios tanto en cliente como en servidor, siendo el backend la autoridad final.
RF-04	El sistema debe registrar fecha/usuario en las operaciones que cambian el estado de un proceso de negocio.
RF-05	El sistema debe implementar el módulo de eventos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-06	El sistema debe implementar el módulo de sesiones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-07	El sistema debe implementar el módulo de sedes/salas con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-08	El sistema debe implementar el módulo de inscripciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-09	El sistema debe implementar el módulo de lista de espera con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-10	El sistema debe implementar el módulo de asistencia con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-11	El sistema debe implementar el módulo de comunicaciones con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-12	El sistema debe implementar el módulo de certificados académicos con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-13	El sistema debe implementar el módulo de feedback con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-14	El sistema debe implementar el módulo de reportes con las operaciones de creación, consulta, actualización o transición que correspondan al dominio, respetando las reglas de negocio definidas.
RF-15	El sistema debe permitir publicar evento válido.
RF-16	El sistema debe permitir respetar cupo y lista de espera.
RF-17	El sistema debe permitir evitar duplicados.
RF-18	El sistema debe permitir registrar asistencia.
RF-19	El sistema debe permitir promover lista de espera.
RF-20	El sistema debe permitir habilitar certificado sólo al cumplir criterio.

H. Requisitos de experiencia web

- CRUD y publicación de eventos
- Calendario
- Inscripciones/lista de espera
- Control de asistencia
- Comunicaciones
- Criterios de certificado

- Dashboard de ocupación y asistencia

La versión web debe priorizar tareas administrativas y operativas de mayor densidad de información. Debe incluir estados de carga, error, vacío, confirmaciones destructivas, validaciones de formulario y navegación consistente.

I. Requisitos de experiencia móvil

- Explorar eventos
- Inscribirse/cancelar
- Mis eventos
- QR/código académico de asistencia
- Notificaciones
- Descargar/visualizar certificado simple
- Facilitador: control de asistencia

La aplicación móvil no debe ser una copia reducida del sitio web. Debe priorizar acciones rápidas, consulta contextual, notificaciones y al menos un flujo transaccional completo relevante para el actor móvil principal.

J. Flujo crítico que debe demostrarse extremo a extremo

Organizador publica evento -> participante se inscribe -> sistema controla cupo/lista de espera -> se registra asistencia -> se habilita certificado según criterio.

K. API REST mínima esperada

Los nombres exactos pueden variar, pero el contrato debe cubrir autenticación, recursos principales, búsquedas y transiciones de estado. No se aceptarán endpoints genéricos del tipo /saveEverything o controladores que expongan directamente entidades JPA.

Ruta conceptual	Método	Propósito
/evento	GET/POST	Consultar/crear evento según permisos
/evento/{id}	GET/PUT/PATCH	Consultar o actualizar evento
/sesionevento	GET/POST	Consultar/crear sesionevento según permisos
/sesionevento/{id}	GET/PUT/PATCH	Consultar o actualizar sesionevento
/sede	GET/POST	Consultar/crear sede según permisos
/sede/{id}	GET/PUT/PATCH	Consultar o actualizar sede
/sala	GET/POST	Consultar/crear sala según permisos
/sala/{id}	GET/PUT/PATCH	Consultar o actualizar sala
/participante	GET/POST	Consultar/crear participante según permisos
/participante/{id}	GET/PUT/PATCH	Consultar o actualizar participante
/inscripcion	GET/POST	Consultar/crear inscripcion según permisos
/inscripcion/{id}	GET/PUT/PATCH	Consultar o actualizar inscripcion
/auth/login	POST	Autenticar y devolver contexto de sesión/token
/reports/...	GET	Consultas agregadas e indicadores del dominio

L. Criterios de aceptación global del proyecto

- Publicar evento válido.
- Respetar cupo y lista de espera.
- Evitar duplicados.
- Registrar asistencia.
- Promover lista de espera.
- Habilitar certificado sólo al cumplir criterio.
- La solución debe iniciar siguiendo instrucciones reproducibles del README y usar migraciones de base de datos.
- Web y móvil deben consumir el mismo backend y respetar permisos y contratos comunes.
- Las operaciones críticas deben tener pruebas y evidencias de error/éxito.

M. Uso acotado de Spring AI

Asistente para generar borradores de descripciones/comunicaciones del evento y resumir feedback de participantes.

La función de IA debe encapsularse detrás de una interfaz/puerto, registrar errores de integración, disponer de timeout/fallback y no bloquear el funcionamiento esencial del MVP. El equipo debe poder explicar qué parte es determinista y qué parte es probabilística.

N. Entregas del proyecto

PARTE I - Parcial 1: problema, requisitos, datos y backend fundacional

Documentación obligatoria del corte:

- Documento de visión y contexto del cliente.
- Matriz completa de actores, objetivos y responsabilidades.
- Catálogo de requisitos funcionales y no funcionales trazables.
- Reglas de negocio numeradas.
- Diagramas de casos de uso o mapa de historias de usuario.
- Modelo conceptual y DER lógico; diccionario de datos inicial.
- Decisiones de arquitectura: modularidad, hexagonal simplificada, convenciones Git y ramas.
- Backlog priorizado y criterios de aceptación del MVP.

Software/MVP obligatorio del corte:

- Repositorio GitHub operativo; README de arranque.
- Docker Compose con PostgreSQL.
- Migraciones iniciales.
- Backend Spring Boot con módulos base, entidades/agregados principales y al menos 3 casos de uso funcionando.
- API inicial documentada y validada con Postman/Bruno/Insomnia.
- React + TypeScript: shell de aplicación, login simulado o real según avance, layout y 1-2 pantallas conectadas.
- React Native + TypeScript: proyecto compilable, navegación base y al menos una pantalla que consuma o simule el contrato API.

Delimitación específica: En PA-10, el equipo debe dejar modelados los conceptos *Evento*, *SesionEvento*, *Sede*, *Sala*, *Participante* y demostrar al menos el alta/consulta de dos recursos principales más el esqueleto del flujo crítico.

PARTE II - Trabajos Prácticos Integradores: flujo transaccional principal

Documentación obligatoria del corte:

- Especificación detallada del flujo principal extremo a extremo.
- Diagramas de secuencia para 2 casos de uso críticos.
- Actualización del DER y diccionario de datos.
- Contrato REST detallado: endpoints, DTOs, códigos HTTP y errores.
- Plan de pruebas del flujo principal.
- Registro de decisiones y deuda técnica conocida.

Software/MVP obligatorio del corte:

- Backend: flujo principal completo con persistencia y validaciones.
- Web: CRUD o flujo transaccional principal completo con formularios, filtros y manejo de errores.
- Móvil: autenticación/navegación y primer flujo transaccional real conectado al backend.
- Pruebas unitarias de reglas críticas.
- Pruebas de integración de al menos un repositorio y un endpoint.
- Pull requests con revisión entre integrantes.

Delimitación específica: El flujo crítico obligatorio para este corte es: Organizador publica evento -> participante se inscribe -> sistema controla cupo/lista de espera -> se registra asistencia -> se habilita certificado según criterio. Debe funcionar al menos hasta el punto central de la transacción, con persistencia real y una vista web y móvil conectadas.

PARTE III - Parcial 2: integración web/móvil, seguridad y calidad

Documentación obligatoria del corte:

- Modelo de permisos por rol.
- Matriz de trazabilidad requisito -> caso de uso -> endpoint -> pantalla -> prueba.
- Documentación de arquitectura actualizada.
- Especificación de estados/transiciones y errores de negocio.
- Plan de pruebas ampliado y evidencias.
- Diseño UX de flujos web y móvil principales.

Software/MVP obligatorio del corte:

- Autenticación/autorización real o académicamente equivalente.
- Backend con la mayoría del alcance funcional obligatorio.
- Web: dashboard/listados/filtros + flujos completos definidos para el proyecto.
- Móvil: al menos dos flujos completos conectados.
- Manejo global de excepciones y validación consistente.
- Paginación/búsqueda donde corresponda.
- CI en GitHub Actions: build y tests de backend, frontend y validación móvil básica.
- Dockerización reproducible.

Delimitación específica: Para PA-10, deben estar operativos los roles Administrador, Organizador, Facilitador, Participante con permisos diferenciados y la mayor parte de los módulos obligatorios: Eventos, Sesiones, Sedes/salas, Inscripciones, Lista de espera, Asistencia, Comunicaciones.

PARTE IV - Examen Final: MVP integrado, IA responsable y defensa

Documentación obligatoria del corte:

- Documento final de requisitos y alcance realmente implementado.
- Manual técnico de instalación/despliegue.

- Manual de usuario breve por rol.
- Catálogo final de API.
- DER final + diccionario de datos final.
- Evidencias de pruebas y matriz de cobertura.
- Registro de riesgos, limitaciones y trabajo futuro.
- Retrospectiva del equipo: organización, conflictos, decisiones y aprendizajes.
- Guion de demostración y defensa individual.

Software/MVP obligatorio del corte:

- MVP web estable y navegable.
- MVP móvil estable con los flujos obligatorios.
- Backend integrado y persistencia consistente.
- Docker Compose de ejecución.
- CI verde.
- Pruebas automáticas esenciales.
- Observabilidad básica y manejo de errores.
- Función Spring AI acotada y con fallback, cuando corresponda al proyecto.
- Datos demo reproducibles.
- Demostración integral extremo a extremo.

Delimitación específica: La entrega final debe demostrar el circuito completo de PA-10, incluyendo reportes/indicadores, trazabilidad, calidad técnica y la función de IA definida de forma segura y secundaria.

O. Preguntas de defensa individual sugeridas

- Explique el flujo crítico desde la pantalla hasta PostgreSQL y de regreso. ¿Qué clases/capas intervienen?
- ¿Qué regla de negocio del proyecto considera más crítica y dónde está implementada? Muestre el código.
- ¿Qué restricción existe también en base de datos y por qué no basta con validarla en React?
- Muestre un endpoint, su DTO, caso de uso, puerto, adaptador de persistencia y prueba asociada.
- ¿Cómo evita el sistema un estado inválido o una transición no permitida?
- ¿Qué diferencia existe entre el modelo de dominio y una entidad JPA en su implementación?
- Muestre un error real del sistema y explique cómo llega un mensaje consistente al cliente web/móvil.
- ¿Qué parte móvil es verdaderamente transaccional y cómo maneja carga, error y reintento?
- ¿Qué prueba automática protege una regla crítica? Modifique temporalmente la regla y explique qué debería fallar.
- ¿Qué decisión técnica cambiaría si el sistema creciera 100 veces y cuál no cambiaría todavía?
- Explique la función de Spring AI: entrada, salida, riesgos, fallback y por qué no sustituye reglas deterministas.
- Abra el historial Git y explique una contribución propia y una contribución de otro integrante que usted también comprenda.

6. Matriz docente de equivalencia de complejidad

Código	Proyecto	Actores	Módulos	Entidades mín.	Patrón de exigencia
PA-01	ClinixFlow	4	9	11	Web + móvil + estados + reglas + reportes + pruebas
PA-02	AutoCare 360	4	10	11	Web + móvil + estados + reglas + reportes + pruebas
PA-03	GastroReserva	4	10	11	Web + móvil + estados + reglas + reportes + pruebas
PA-04	CondoFlow	4	9	11	Web + móvil + estados + reglas + reportes + pruebas
PA-05	EduMentor	3	10	11	Web + móvil + estados + reglas + reportes + pruebas
PA-06	StockFlow	4	10	10	Web + móvil + estados + reglas + reportes + pruebas
PA-07	FixDesk	4	10	11	Web + móvil + estados + reglas + reportes + pruebas
PA-08	FitClub	4	10	10	Web + móvil + estados + reglas + reportes + pruebas
PA-09	PetCare Manager	4	10	11	Web + móvil + estados + reglas + reportes + pruebas
PA-10	EventHub	4	10	11	Web + móvil + estados + reglas + reportes + pruebas

La equivalencia no se mide por contar pantallas, sino por la presencia de un flujo transaccional con estados, reglas de negocio, persistencia relacional, permisos, experiencia web, experiencia móvil, pruebas, DevOps y una defensa individual del sistema completo.

7. Definition of Done común por historia funcional

- La historia tiene criterio de aceptación verificable y requisito/regla asociados.
- El modelo de datos y migración están actualizados cuando corresponde.
- El backend valida entradas y reglas; no delega reglas críticas al frontend.
- El endpoint usa DTOs y códigos HTTP coherentes.
- La UI web o móvil muestra carga, éxito, vacío y error según corresponda.
- Existe al menos una prueba automática cuando la historia contiene lógica relevante.
- No quedan credenciales ni secretos en el repositorio.
- El código fue integrado mediante pull request y revisado por otro integrante.
- La documentación afectada fue actualizada.
- Un integrante distinto del autor puede explicar y demostrar la funcionalidad.

8. Lista de control del docente por entrega

Dimensión	Pregunta de control
Requisitos	¿El equipo implementa lo pedido sin reducir unilateralmente el alcance?
Datos	¿El DER, restricciones y migraciones corresponden al dominio real?
Backend	¿Las reglas están en casos de uso/dominio y existe separación razonable por módulos?
API	¿Contratos, DTOs, errores y estados HTTP son consistentes?
Web	¿Existe incremento funcional conectado, no sólo maquetas?
Móvil	¿Existe incremento funcional conectado desde etapas tempranas?
Calidad	¿Pruebas, manejo de errores y trazabilidad son verificables?
Git/CI	¿Hay colaboración real, PRs, revisión y pipeline reproducible?
Documentación	¿Los artefactos describen el sistema implementado y no un sistema imaginario?
Defensa	¿Cada integrante puede explicar el avance completo del corte?